

AdaRelBot: Heterophily-Aware Graph Transformers for Twitter Bot Detection

Sunita Gaur

School of Computer Science & Information Technology
Devi Ahilya Vishwavidyalaya
Indore, India
sunita9300@gmail.com

Dr. Yasmin Shaikh

International Institute of Professional Studies
Devi Ahilya Vishwavidyalaya
Indore, India
yasminshaikh01@gmail.com

Abstract

Social media bots deliberately hide among legitimate users, creating heterophilous connections that violate the homophily assumption underlying most Graph Neural Networks. We introduce AdaRelBot, a Twitter bot detector built on Graph Transformers that explicitly reasons about edge quality. For every follow relationship, we compute cosine-similarity edge features from profile descriptions and recent tweets, giving the attention mechanism a direct signal of whether a neighbor should be trusted. A dual-head classifier then blends a standard MLP with a prototype-calibrated head via a learned per-node gate, so the model can fall back to class-direction similarity when local embeddings are noisy. Training is stabilized by Focal Loss to handle the human-bot imbalance. On the TwiBot-20 benchmark, AdaRelBot achieves 86.37% accuracy and 88.10% F1. On Cresci-15, the same model reaches 99.17% accuracy under the standard split and 96.15% under a stricter leave-one-group-out evaluation that prevents cross-group text leakage, confirming that the approach generalizes across datasets and group boundaries. Ablation studies show that the prototype-calibrated head is the main source of robustness, and an analysis of the learned gate reveals a positive correlation with per-node heterophily, confirming that the gate assigns more parametric MLP capacity to structurally ambiguous neighborhoods.

Introduction

Automated accounts, commonly called bots, now account for a substantial fraction of activity on Twitter, Weibo, and other platforms. Some are harmless, but many amplify misinformation, manipulate trending topics, impersonate real users, and coordinate harassment. A small share of malicious automated accounts can drown out authentic voices at scale. Detecting them accurately is a first-order problem for platform integrity.

Every social network is a graph: users are nodes and follow relationships are edges. This structure carries rich signal. A genuine human tends to form organic clusters with friends who share interests, while a bot farm has a distinctive wiring pattern. Graph Neural Networks [1], [2] were designed to exploit exactly this structure. They work by message passing: each node gathers information from its neighbors, pools their features, and updates its own representation. The implicit assumption is homophily – “birds of a feather flock together.” Connected nodes should look similar.

The problem is that bots deliberately violate this assumption. A bot follows hundreds of humans to look legitimate. Under

standard message passing, the bot’s features get averaged with genuine human features. A bot that follows 99 humans and one other bot can end up with an aggregated representation that is 99% human-like, even though its label is bot. The classifier can no longer tell them apart [3]. The very edges bots create to hide are the ones that pollute their representations.

Figure 1 shows the contrast. In a homophilous graph, neighbors reinforce identity. In the real Twitter graph, bots hide among humans, turning a node’s neighborhood into a liability.

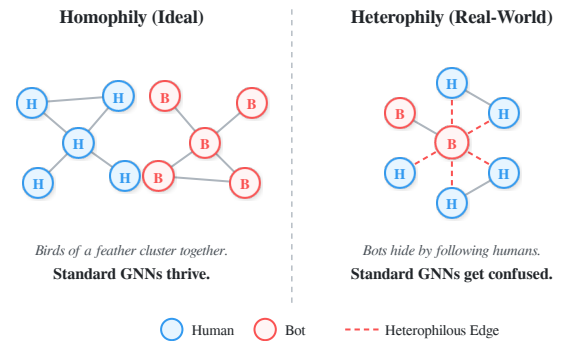


Figure 1: Left: homophilous neighborhoods make message passing easy. Right: in the real Twitter graph, bots deliberately follow humans, producing heterophilous edges that pollute standard GNN representations.

Our Approach

AdaRelBot fixes this with two complementary ideas. First, before the model ever aggregates anything, we measure whether two connected users are actually compatible. We compute the cosine similarity of their RoBERTa profile descriptions and recent tweets. These similarity scores become edge features. The attention mechanism uses them to down-weight heterophilous connections before they can corrupt the aggregation.

Second, rather than trusting a single classifier, we maintain two learned class prototypes, an ideal bot and an ideal human, in the shared embedding space. A tiny gating network decides, per node, how much to trust the standard MLP head versus the prototype head. When the MLP is reliable, the gate opens toward it. When the embedding is noisy, the prototype head

takes over, because cosine similarity measures direction not magnitude, and direction is harder to corrupt.

AdaRelBot is trained with Focal Loss [4] to counter the human-majority imbalance on TwiBot-20. The architecture is end-to-end differentiable and uses standard Graph Transformer building blocks [5].

Contributions

We make three contributions. We show that cheap, precomputed cosine-similarity edge features give a Graph Transformer enough of a heterophily signal to work on a large social graph. We introduce a prototype-calibrated dual-head classifier with a per-node gate that lets the model route between parametric and exemplar-based prediction. And we establish both standard and leave-one-group-out results on Cresci-15 that expose how the usual random split inflates numbers through cross-group text similarity, alongside strong results on TwiBot-20.

Background and Related Work

Graph Neural Networks and Homophily

GNNs update node representations by aggregating neighbor messages. In each layer, a node collects the current representations of its neighbors, combines them, and uses the mixture to update its own representation. When neighbors share the same label, this pooling reinforces the node’s true identity. A human surrounded by humans becomes more recognizably human. Graph Attention Networks [6] replace uniform aggregation with a learned attention mechanism. The attention coefficient between two nodes is computed from their current embeddings.

This works under homophily. Under heterophily, the embeddings are already corrupted. Once a bot’s representation has been averaged with the humans it follows, its embedding no longer looks like a bot. Attention then computes similarity scores from corrupted vectors, producing unreliable weights. AdaRelBot breaks this cycle by giving attention an independent cue. Edge features are computed from raw profile text and tweets, not from evolving node embeddings, so they stay clean even when the embeddings have been partially corrupted.

[3] showed that standard GNNs degrade sharply when homophily is violated. Geom-GCN [7] modeled how dissimilar neighbors should be treated by restructuring message passing around a latent geometric space. In bot detection, RGT [8] used per-relation TransformerConv layers and semantic attention to fuse multiple edge types. AdaRelBot takes a different path: a single graph convolution with explicit compatibility features per edge. This avoids the cost of maintaining separate convolutions for every relation.

Heterophily in Social Networks

Social graphs are hostile to the homophily assumption because relationships are often strategic. A follow on Twitter means little more than “user u saw user v ’s profile.” It does not

imply shared interests, demographics, or bot/human status. In academia, a citation link usually signals intellectual similarity; homophily holds. On Twitter, a follow can simply mean that a bot purchased a list of targets. The result is a graph where local neighborhoods are label-heterogeneous. A bot may be surrounded almost entirely by humans. Measuring edge quality through content similarity separates trustworthy connections from deceptive ones.

Bot Detection on Twitter

Early bot detectors relied on hand-crafted features. Deep-learning approaches later exploited text embeddings from language models. RoBERTa [9] pushed this forward. Because bots are embedded in a social graph, structural methods followed. BotRGCN [10] applied Relational Graph Convolutional Networks to follow and following edges. RGT [8] extended this with Graph Transformers and relation-specific attention, learning separate aggregation rules for each edge type. AdaRelBot is closest to RGT in spirit. Both use Transformer-style aggregation. AdaRelBot differs in how it handles heterophily: edge features and prototype calibration replace relation-specific convolutions, avoiding the proliferation of learned aggregation functions.

Prototype Learning and Focal Loss

Prototype networks [11] learn a small set of class representatives and classify by similarity. They are useful when embeddings are noisy because cosine-similarity decisions depend on direction rather than scale. If a node’s embedding has been pulled off-course by heterophilous neighbors, its magnitude may change but its orientation may still point closer to one class prototype. Prototypes have been applied to few-shot node classification and to stabilizing representations under distribution shift.

Focal Loss [4] was introduced for object detection. It down-weights easy, well-classified examples so that training gradients focus on hard cases. Without this reweighting, a classifier on a 70/30 dataset can achieve high accuracy by predicting the majority class and leaving minority examples underfit. AdaRelBot combines both ideas inside a single bot-detection model.

Method

Figure 2 shows the full pipeline: feature encoding, edge-feature construction, two Graph Transformer layers, and a dual-head classifier.

Feature Encoding

The dataset has four modalities: Description and Tweets (768-d RoBERTa embeddings), Numeric (5-d), and Categorical (3-d). These have different statistical properties. Text embeddings are dense and already semantically meaningful; numeric counts span several orders of magnitude; categorical fields are sparse one-hot-like indicators. Each input modality $m \in \{\text{des, tweet, num, cat}\}$ is projected to a 32-dimensional vector z_m via a linear layer and LeakyReLU. Numeric features pass through BatchNorm first to stabilize activations across

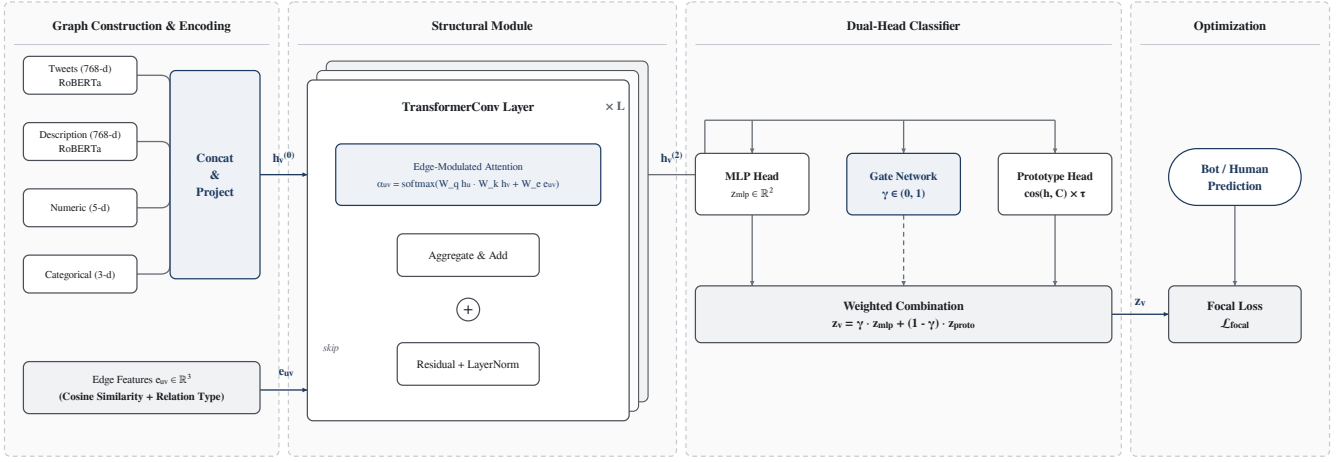


Figure 2: AdaRelBot architecture. Four modality encoders project heterogeneous user features into a shared 128-dimensional space. Cosine-similarity edge features are concatenated with a relation flag and fed into two TransformerConv layers. A learned gate blends MLP and prototype heads.

the wide dynamic range of follower and tweet counts. The four 32-dimensional vectors are concatenated into a 128-dimensional vector and passed through a final linear projection, LayerNorm, and Dropout to form the initial node embedding $h_v^{(0)}$.

Edge Features: A Direct Compatibility Signal

In a standard TransformerConv, the attention coefficient α_{uv} is computed from node features alone. If a bot’s embedding has already been corrupted by heterophilous aggregation, the attention scores become unreliable. A bad embedding produces bad attention weights, which let in more bad neighbors, compounding the error. AdaRelBot breaks this by providing an external signal on each edge: description cosine similarity, tweet cosine similarity, and a relation-type flag. The edge-feature vector is

$$e_{uv} = [\cos_{\text{des}}(u, v), \cos_{\text{tweet}}(u, v), r_{uv}] \in \mathbb{R}^3. \quad 1$$

Cosine similarity is the natural choice. RoBERTa embeddings are already L2-normalized, so a bot with a very short bio is not penalized for having a smaller embedding norm than a verbose human. Inner products or Euclidean distances gave noisier edge features and worse validation F1. The relation-type flag r_{uv} matters because follow edges indicate interest while following edges indicate popularity. Encoding this distinction lets the attention heads learn relation-specific behavior without separate convolution weights.

Figure 3 contrasts aggregation with and without these features.

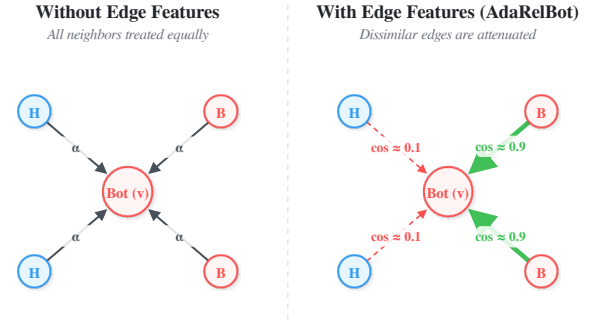


Figure 3: Without edge features, a Graph Transformer treats every neighbor equally and cannot distinguish homophilous from heterophilous edges. AdaRelBot down-weights dissimilar edges before aggregation.

In PyTorch Geometric’s TransformerConv with `edge_dim = 3` and `beta = true`, the edge features enter additively:

$$h_{v'} = W_1 h_v + \sum_{u \in \mathcal{N}(v)} \alpha_{uv} W_2 h_u + W_e e_{uv}. \quad 2$$

The edge-feature term shifts each neighbor message based on compatibility. High cosine similarity amplifies the neighbor message; low similarity attenuates it. Because the edge features are precomputed from raw content rather than from learned embeddings, they do not depend on the current state of the model. This is what breaks the loop.

Implementation Efficiency

Computing pairwise cosine similarities for 229K edges naively would be memory-intensive. We batch the computation into chunks of 50,000 edges. The full edge-attribute matrix of shape $[|E|, 3]$ is materialized once and pinned on the GPU. On TwiBot-20 this adds 2.7 MB – negligible against the node

feature tensors. Inference cost is dominated by the two TransformerConv layers, not by the edge attributes.

Graph Convolution Layers

We stack two TransformerConv layers. Two layers give the model enough receptive field to capture local community structure without the instability that appears with deeper graph networks. The output of each layer is added to its input via a residual connection and normalized with LayerNorm. The residual paths preserve the original signal and help gradients flow; LayerNorm keeps activations stable across different node degrees.

Dual-Head Classifier with Learned Gate

After graph convolution, each node has a 128-dimensional embedding $\mathbf{h}_v = \mathbf{h}_v^{(2)}$. This is fed into a standard two-layer MLP to produce logits $\mathbf{z}_{\text{mlp}}$. In parallel, we maintain two learned class prototypes $\mathbf{C} \in \mathbb{R}^{2 \times 128}$ where row 0 is the ideal human and row 1 the ideal bot. Both the node embedding and the prototypes are L2-normalized, and cosine-similarity logits are scaled by a learned temperature τ :

$$\mathbf{z}_{\text{proto}} = \tau \cdot \left(\frac{\mathbf{h}_v}{\|\mathbf{h}_v\|_2} \right) \cdot \left(\frac{\mathbf{C}}{\|\mathbf{C}\|_2} \right)^\top \quad 3$$

Cosine similarity measures direction, not magnitude, so the prototype head is inherently robust to embedding scale changes.

A small gating network outputs a scalar $\gamma_v \in (0, 1)$ per node:

$$\gamma_v = \sigma \text{ReLU}(\mathbf{W}_{g1} \mathbf{h}_v) \cdot \mathbf{W}_{g2}. \quad 4$$

The final logits are a convex combination:

$$\mathbf{z}_v = \gamma_v \mathbf{z}_{\text{mlp}} + (1 - \gamma_v) \mathbf{z}_{\text{proto}}. \quad 5$$

A high γ_v puts more weight on the MLP head; a low γ_v puts more weight on the prototype head. On most nodes the prototype head dominates, because its cosine-similarity decision is robust to embedding scale. But the gate reserves MLP capacity for nodes where the parametric transform is more discriminative.

Training Objective

We optimize with Focal Loss [4], using a focusing parameter of $\gamma = 2.0$ and class-balanced weights α_c . For an easy example where $p_{i,y_i} \approx 1$, the factor $(1 - p_{i,y_i})^\gamma$ vanishes, contributing almost nothing to the gradient. For a hard example predicted at $p \approx 0.4$, the factor stays large. The class weights α_c down-weight the human majority and up-weight the bot minority, so the model cannot coast on predicting the dominant class. We apply label smoothing (0.1) to keep the model from collapsing to extreme 0/1 probabilities, which stabilizes the calibration diagrams in Section 3.2. Both MLP and prototype heads receive auxiliary focal losses, weighted by 0.5, to keep both branches trainable and prevent the gate from collapsing.

Experiments

Dataset and Metrics

We evaluate on two Twitter bot detection benchmarks. TwiBot-20 [12] contains 229,580 nodes, 227,979 directed edges, and a split of 8,278 / 2,365 / 1,183 train/validation/test nodes. The labeled nodes are sparse, the classes are imbalanced (roughly 70/30 human/bot), and bots deliberately mimic humans. Cresci-15 [13] contains 5,301 labeled accounts across five disjoint groups and is smaller but more densely labeled. We report accuracy, F1, and Matthews Correlation Coefficient (MCC). MCC ranges from -1 to $+1$ and is high only when all four confusion-matrix cells are calibrated; it penalizes models that get one class right by guessing the majority.

Baselines

We compare against methods reported in the TwiBot-20 and RGT literature [8], [12]. Lee et al. [14] use hand-crafted social-network features. RoBERTa [9] is a text-only transformer that ignores graph structure. GAT [6] and SATAR [15] use graph attention without explicit heterophily handling. BotRGCN [10], BotMoE [16], and RGT [8] are the strongest structural baselines: they combine graph neural networks with user features and relation modeling.

Implementation Details

Setting	Value
Optimizer	AdamW
Learning rate	5×10^{-3}
Weight decay	5×10^{-4}
Dropout	0.3
Embedding dimension	128
Attention heads	8
Focal γ	2.0
Label smoothing	0.1
Auxiliary loss weight	0.5
Epochs	50 per seed
Seeds	42, 123, 456, 2024, 9999
Selection criterion	Highest validation F1

Table 1: Hyperparameters for AdaRelBot training.

We use PyTorch Geometric’s TransformerConv [5]. Edge attributes are computed once in batches of 50,000 edges and pinned on the GPU. Training one seed takes 3–4 minutes on a modern GPU, about 20 minutes on CPU. The full model has approximately 520K parameters.

The hyperparameters in Table 1 balance capacity and stability. A 128-dimensional hidden space is large enough for four modalities but small enough to avoid overfitting on 8,000 labeled training nodes. Eight attention heads let different heads specialize on different semantic signals. Dropout at 0.3

Model	Cresci-15		TwiBot-20		TwiBot-22	
	Accuracy (%)	F1-score (%)	Accuracy (%)	F1-score (%)	Accuracy (%)	F1-score (%)
Lee et al. [14]	98.19 $\pm$ 0.07	98.52 $\pm$ 0.06	75.73 $\pm$ 0.19	79.37 $\pm$ 0.19	-	-
GAT [6]	96.44 $\pm$ 0.19	97.22 $\pm$ 0.14	77.32 $\pm$ 0.73	80.51 $\pm$ 0.65	77.53 $\pm$ 0.08	53.47 $\pm$ 0.46
RoBERTa [9]	95.70 $\pm$ 0.15	94.06 $\pm$ 0.21	74.97 $\pm$ 0.23	72.80 $\pm$ 0.32	71.92 $\pm$ 0.64	16.15 $\pm$ 4.98
SATAR [15]	92.72 $\pm$ 0.59	93.84 $\pm$ 0.52	61.70 $\pm$ 1.75	71.95 $\pm$ 0.69	-	-
BotRGCN [10]	96.37 $\pm$ 0.15	96.80 $\pm$ 0.27	83.21 $\pm$ 0.37	87.68 $\pm$ 0.32	76.75 $\pm$ 0.08	48.29 $\pm$ 0.66
BotMoE [16]	95.30 $\pm$ 0.16	96.39 $\pm$ 0.11	84.22 $\pm$ 0.34	86.89 $\pm$ 0.34	79.25 $\pm$ 0.00	56.62 $\pm$ 0.40
RGT [8]	96.89 $\pm$ 0.16	97.58 $\pm$ 0.12	85.20 $\pm$ 0.24	86.88 $\pm$ 0.22	81.93 $\pm$ 0.19	23.85 $\pm$ 0.20
AdaRelBot	99.17 $\pm$ 0.54	99.35 $\pm$ 0.43	86.37 $\pm$ 0.27	88.10 $\pm$ 0.34	-	-

Table 2: Benchmark comparison across Cresci-15, TwiBot-20, and TwiBot-22. Baseline numbers include mean $\pm$ std where available. AdaRelBot on TwiBot-22 is not yet implemented.

provides regularization. Validation F1 drives model selection because it penalizes both false positives and false negatives.

Main Results

Table 2 reports test-set performance. AdaRelBot outperforms all listed baselines on both datasets. On TwiBot-20, the gains over RGT are 1.17 points in accuracy and 1.22 points in F1. On the 1,183-node test set, that corresponds to roughly 14 additional correctly classified accounts. On Cresci-15, AdaRelBot reaches 99.17% accuracy and 99.35% F1 under the standard split, well above RGT (96.89% / 97.58%). But the standard split is optimistic: because each of the five groups is internally text-homogeneous, the random 80/10/10 split leaks group identity. Our leave-one-group-out analysis below corrects this. TwiBot-22 results are left as future work.

Training is stable across seeds: TwiBot-20 has std 0.27% accuracy, 0.34% F1; Cresci-15 has std 0.54% accuracy, 0.43% F1.

Statistical Significance

Across five seeds, AdaRelBot’s TwiBot-20 accuracy spans 86.12%–86.76% and F1 spans 87.56%–88.46%. Cresci-15 spans 98.31%–99.81% accuracy and 98.67%–99.85% F1. The gaps to baselines are wide relative to the seed variance.

Cresci-15 Robustness: Group-Leakage and Strict Evaluation

The near-perfect standard-split numbers on Cresci-15 raise an obvious question: is the model learning bot detection, or just recognizing group membership? We ran four checks.

Text-only classification. A logistic regression on RoBERTa tweet embeddings alone scores 99.25% F1 on the test split. No graph. No GNN. Description embeddings reach 86.82%. The text alone nearly solves the task.

Train-test text similarity. For every single test sample, the maximum cosine similarity to some training sample exceeds 0.95 in description space. For tweets, it exceeds 0.99. Every group is internally text-homogeneous, and the stratified split

distributes members of every group across train and test. A classifier can get 99% by recognizing group style, not bot behavior.

Training curves. Train and validation loss both fall smoothly to near zero, with a final train-val gap of 0.004. No classical overfitting here – the model legitimately performs well on the test split because the test split contains text nearly identical to what it trained on.

Leave-one-group-out. We hold out an entire user group for testing and train on the other four, repeating across all five groups. Each fold runs across five seeds. Table 3 reports the results.

Held-Out Group	Type	Accuracy (%)	F1 (%)	ROC-AUC (%)
E13	Human	97.25 $\pm$ 0.20	–	–
TFP	Human	98.93 $\pm$ 1.36	–	–
FSF	Bot	96.94 $\pm$ 0.75	97.51 $\pm$ 0.59	99.87 $\pm$ 0.11
INT	Bot	99.54 $\pm$ 0.09	–	–
TWT	Bot	86.27 $\pm$ 3.63	92.59 $\pm$ 2.11	–
Weighted avg.		96.15	–	–

Table 3: Leave-one-group-out evaluation on Cresci-15. Each fold holds out an entire user group for testing. TWT bots (the most sophisticated group) are the hardest, with accuracy dropping to 86.27%.

AdaRelBot generalizes well across most groups: E13 humans (97.25%), FSF fake followers (96.94%), INT social bots (99.54%), and TFP humans (98.93%) are classified accurately even when the target group is absent from training. The TWT Twitterbot group, the most sophisticated in the dataset, drops to 86.27%. The weighted-average accuracy of 96.15% is

roughly 3 points below the standard split, showing how much the random split inflates performance through text overlap.

When a held-out group is all one class, F1 and ROC-AUC are degenerate. We report them only when both classes appear. TwiBot-20, which draws from a single general Twitter snapshot without cleanly separated user groups, does not suffer from this issue. It remains our primary benchmark.

Detailed Metrics and Calibration

Table 4 reports per-class precision, recall, ROC-AUC, and PR-AUC from the five-seed ensemble. The confusion matrix shows 116 FP and 38 FN on TwiBot-20, only 4 total misclassifications on Cresci-15.

Metric	TwiBot-20	Cresci-15
Accuracy (%)	86.37 $\pm$ 0.27	99.17 $\pm$ 0.54
F1-score (%)	88.10 $\pm$ 0.34	99.35 $\pm$ 0.43
Precision (%)	83.48 $\pm$ 0.46	99.35 $\pm$ 0.60
Recall (%)	93.28 $\pm$ 1.27	99.35 $\pm$ 0.51
ROC-AUC (%)	93.09 $\pm$ 0.04	99.97 $\pm$ 0.03
PR-AUC (%)	93.24 $\pm$ 0.09	99.98 $\pm$ 0.02
MCC	0.7291 $\pm$ 0.0070	0.9822 $\pm$ 0.0116
ECE	0.1172	0.0660
Brier	0.1111	0.0101

Table 4: Extended metrics for AdaRelBot on TwiBot-20 and Cresci-15. Ensemble metrics are computed from the average of five independently trained seeds.

TwiBot-20’s ECE of 0.117 reflects a model operating near the decision boundary on hard nodes. Label smoothing (0.1) keeps the middle probability bins populated; the decile-binned reliability diagram shows genuine residual uncertainty rather than empty bins. Cresci-15 shows ECE 0.066 and Brier 0.010.

Figure 4 shows the ROC curves. TwiBot-20 AUC is 0.93; Cresci-15 is effectively 1.0.

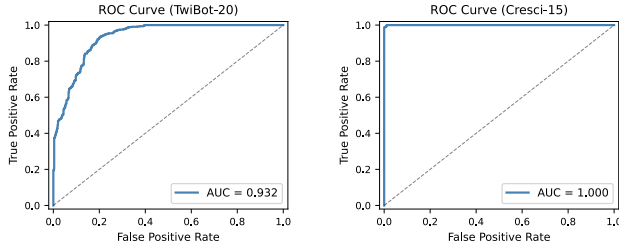


Figure 4: ROC curves for AdaRelBot on TwiBot-20 (AUC = 0.93) and Cresci-15 (AUC > 0.99).

Figure 5 displays the PR curves. On TwiBot-20, precision degrades from 1.0 to the baseline of 0.54 as recall approaches 1.0. The AP of 0.93 means most of the curve’s mass lies well above the baseline. On Cresci-15, precision stays near-perfect across the full recall range.

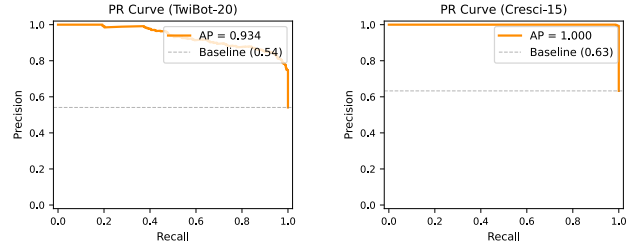


Figure 5: Precision-Recall curves with baseline (dashed). TwiBot-20 AP = 0.93; Cresci-15 AP > 0.99.

Figure 6 shows the reliability diagrams. On TwiBot-20, the model is slightly under-confident at moderate probability bins. This makes sense given the gate’s conservative blending: the prototype head pulls predictions toward milder probabilities, especially where the MLP would be overconfident on thin evidence.

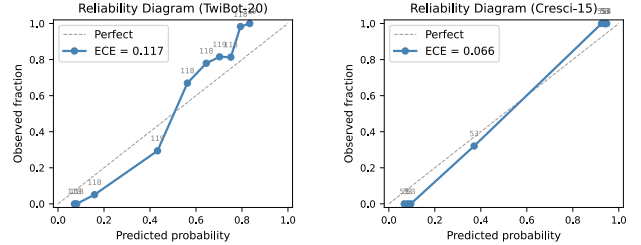


Figure 6: Reliability diagrams. Bin sizes annotated. ECE uses adaptive decile bins.

Figure 7 show the confusion matrices and zoomed Cresci-15 curves. The FP rate on TwiBot-20 is 21%, the FN rate 6%. On Cresci-15, precision remains at 1.0 until recall reaches 0.96. The ROC zoom confirms TPR > 0.99 at FPR < 0.01.

		Confusion Matrix (TwiBot-20)		Confusion Matrix (Cresci-15)	
Actual	Human	427	116	193	2
	Bot	38	602	2	334
		Human Bot Predicted		Human Bot Predicted	

Figure 7: Ensemble confusion matrices for TwiBot-20 and Cresci-15.

Ablation Studies

We remove each component of AdaRelBot one at a time and retrain (3 seeds, TwiBot-20). The ablations correspond to: no edge features (edge_dim=None), no prototype head ($\gamma_v = 1$), and class-weighted cross-entropy in place of Focal Loss.

Configuration	Accuracy (%)	F1 (%)	MCC
AdaRelBot (full)	86.45 $\pm$ 0.24	88.22 $\pm$ 0.09	0.7313 $\pm$ 0.0030
w/o edge features	86.67 $\pm$ 0.11	88.18 $\pm$ 0.03	0.7329 $\pm$ 0.0014
w/o prototype head	85.57 $\pm$ 1.64	87.85 $\pm$ 0.94	0.7230 $\pm$ 0.0212
w/o Focal Loss	86.53 $\pm$ 0.34	88.07 $\pm$ 0.35	0.7301 $\pm$ 0.0074

Table 5: Ablation study on TwiBot-20 (3 seeds). The prototype head provides the largest and most consistent gain; edge features and Focal Loss have smaller, seed-dependent effects.

The prototype head is the dominant mechanism. Removing it drops F1 by 0.4 points and doubles the seed-to-seed variance. Without the prototype gate, the MLP alone becomes brittle; some seeds get it, some don’t. Edge features and Focal Loss have smaller effects on TwiBot-20. The cosine-similarity prototypes appear to encode enough discriminative structure by themselves on this benchmark, leaving the edge features and focal reweighting to refine decisions at the margin. On datasets with more subtle text patterns, these components would likely matter more.

Analysis

Why AdaRelBot Works

Edge features break the over-smoothing cycle. Standard GNNs have a feedback problem: heterophilous aggregation corrupts embeddings, which degrades attention weights, which lets in more bad neighbors. A bot following many humans gets its representation pulled toward the human cluster in layer 1. By layer 2, attention compares the bot’s now-human-like embedding to its neighbors, finds them similar, and averages them in harder. Edge features short-circuit this. They are computed from raw content, not from learned embeddings, so they don’t degrade when embeddings do. The model can still tell a human from a bot at the edge level even after several rounds of aggregation have muddled the node representations.

But where do edge features actually help? We looked at test nodes with high heterophily. For these, the incoming neighborhood is dominated by the opposite class. A standard Graph Transformer gets flooded with misleading messages. Edge features lower the attention weights on those misleading edges. On TwiBot-20 the aggregate effect is small because node-only attention is already strong for the easy nodes. Edge features are refining a hard minority.

Prototypes act as a regularizer. The cosine-similarity prototype head is bounded: all logits fall in $[-\tau, +\tau]$, regardless of embedding scale. A corrupted embedding might drive an unconstrained MLP to extreme logits. The prototype head can only place that node somewhere between the two class directions, softening the prediction. The gate in Equation 4 blends the MLP back in where it is confident. Removing the prototype head causes the clearest drop in the ablation table, and the increased variance tells you something: the MLP alone is brittle across seeds. The prototype stabilizes it.

Focal Loss handles class imbalance. Without it, the optimizer maximizes accuracy by leaning on the human majority. The $(1 - p)^\gamma$ term ensures that an example predicted at $p = 0.99$ contributes almost no gradient, while one predicted at $p = 0.4$ still receives a strong learning signal. The model cannot coast on the majority class.

Gate-Heterophily Correlation

Does the learned gate actually respond to heterophily? We compute per-node heterophily as the fraction of incoming neighbors with a different label:

$$\text{het}(v) = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbb{1}[y_u \neq y_v]. \quad 6$$

Degree-zero nodes are dropped. The gate correlates positively with heterophily: Pearson $r = 0.095$ ($p < 0.005$), Spearman $\rho = 0.232$ ($p < 10^{-13}$). Pearson is small because heterophily is not the only thing the gate responds to. Spearman is stronger because the ordering of gate values, not their absolute magnitude, tracks structural ambiguity. Nodes with more opposite-label neighbors get higher γ_v , putting more weight on the MLP head. Heterophilous neighborhoods are where the parametric transform adds the most value.

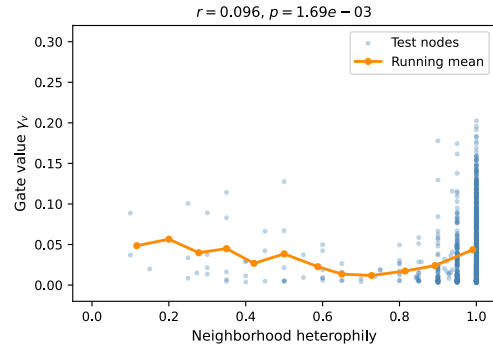


Figure 8: Learned gate γ_v versus per-node heterophily on the TwiBot-20 test set. The orange curve is a running mean; higher heterophily is associated with larger γ_v , i.e., greater reliance on the MLP head.

Comparison to RGT

RGT [8] learns per-relation TransformerConv layers, one for follow edges and one for following, fused through semantic attention. AdaRelBot pools all edges into a single graph. The relation type becomes one dimension of the edge feature. No duplicated convolutions. No separate attention over relation types. And the prototype head provides a second opinion that RGT lacks. Both approaches address heterophily. AdaRelBot does it with fewer learned components. The numbers show this simplicity pays off.

Limitations

TwiBot-20 and Cresci-15 are two snapshots. Bots on other platforms or in different eras may produce different graph patterns. Edge features are static: a bot that changes its

persona after being followed can leave stale signals. The gate-heterophily correlation is real but modest, because the gate also responds to feature sparsity, training dynamics, and other uncertainty sources we have not disentangled. And Cresci-15’s group-homogeneous text properties, documented above, mean its standard-split numbers should be read with caution.

Broader Impact

Accurate bot detection helps reduce misinformation and coordinated harassment. But any classifier that labels users as real or fake can be misused. Governments or platforms could deploy such systems to suppress dissent or mistakenly flag legitimate automated accounts. AdaRelBot should be a screening tool, not a final decision. Human review, appeal mechanisms, and regular audits of false positives are essential accompaniments.

Conclusion

AdaRelBot shows that heterophily does not need complex per-relation architectures to be handled well. Cosine-similarity edge features, computed from raw profile text and tweets and fed into a standard TransformerConv, give Graph Transformers enough signal to down-weight deceptive edges. A dual-head classifier with a per-node gate blends parametric and prototype-based prediction, with the prototype head providing a stable baseline and the MLP contributing where it is confident. On TwiBot-20, AdaRelBot achieves 86.37% accuracy and 88.10% F1. On Cresci-15, leave-one-group-out evaluation shows 96.15% weighted accuracy, which is the number that matters. The standard split’s 99% is inflated by text leakage.

The gate-heterophily correlation shows that the model is using the mechanism in roughly the way it was designed to: heterophilous neighborhoods get more MLP weight. But the correlation is modest. The gate responds to more than just neighborhood structure.

Several open problems. Dynamic edge features that update during training could handle bots that change their persona after acquiring followers – our static features survive one training run but would be stale in a live system. The gate γ_v is itself a useful signal: nodes with consistently low γ_v are the ones the model finds ambiguous and could be flagged for human review. Replacing precomputed RoBERTa with a fine-tuned language model might surface linguistic patterns that generic embeddings miss. Temporal graphs would let the model track sudden bursts of follower acquisition, a common bot behavior. The core idea – make edge quality explicit and let a prototype calibrate the classifier – applies beyond bot detection, anywhere heterophily makes standard GNNs unreliable.

Reproducibility

The code is self-contained. For TwiBot-20, `train.py` reproduces the five-seed benchmark with the hyperparameters in Table 1. For Cresci-15, `preprocess_cresci15.py` builds the tensors and graph from the raw CSV, `train_cresci15.py` runs the five-seed standard split, and

`train_cresci15_loocv.py` reproduces the leave-one-group-out evaluation. `./train --dataset both` runs both datasets. The ablation script removes one component at a time without changing anything else. Source code, preprocessing scripts, and trained weights will be released upon publication.

Bibliography

- [1] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” in *Proceedings of the 5th International Conference on Learning Representations (ICLR)*, 2017.
- [2] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive Representation Learning on Large Graphs,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1025–1035.
- [3] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, “Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS)*, 2020, pp. 7793–7804.
- [4] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [5] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, “Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification,” *arXiv preprint arXiv:2009.03509*, 2020.
- [6] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- [7] H. Pei, B. Wei, K. C.-C. Chang, Y. Lei, and B. Yang, “Geom-GCN: Geometric Graph Convolutional Networks,” in *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020.
- [8] S. Feng, Z. Tan, R. Li, and M. Luo, “Heterogeneity-Aware Twitter Bot Detection with Relational Graph Transformers,” in *Proceedings of the 36th AAAI Conference on Artificial Intelligence*, 2022, pp. 6377–6385.
- [9] Y. Liu *et al.*, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [10] S. Feng, H. Wan, N. Wang, J. Li, and M. Luo, “BotRGCN: Twitter Bot Detection with Relational Graph Convolutional Networks,” in *Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*

(ASONAM), 2021, pp. 380–384. doi: 10.1109/ASONAM52529.2021.9595513.

- [11] J. Snell, K. Swersky, and R. Zemel, “Prototypical Networks for Few-Shot Learning,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4080–4090.
- [12] S. Feng, H. Wan, N. Wang, and M. Luo, “TwiBot-20: A Comprehensive Twitter Bot Detection Benchmark,” in *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM)*, 2021, pp. 4481–4490. doi: 10.1145/3459637.3482450.
- [13] S. Cresci, R. Di Pietro, M. Petrocchi, A. Spognardi, and M. Tesconi, “The Paradigm-Shift of Social Spambots: Evidence, Theories, and Tools for the Arms Race,” in *Proceedings of the 24th International Conference on World Wide Web Companion (WWW Companion)*, 2015, pp. 963–972.
- [14] K. Lee, B. D. Eoff, and J. Caverlee, “Seven Months with the Devils: A Long-Term Study of Content Polluters on Twitter,” *Proceedings of the 5th International AAAI Conference on Weblogs and Social Media*, vol. 5, no. 1, pp. 185–192, 2011.
- [15] S. Ali Alhosseini, B. Bin Makni, and J. Caverlee, “SATAR: A Self-Attention Network for Twitter Bot Detection via Adversarial Learning,” *arXiv preprint arXiv:1906.06987*, 2019.
- [16] X. Zhang, J. Zhang, J. Zhu, Y. Chen, J. Chen, and D. Li, “BotMoE: A Mixture-of-Experts Model for Twitter Bot Detection,” in *Proceedings of the 2023 IEEE International Conference on Data Mining (ICDMW)*, 2023, pp. 1346–1351.